SmartPaisa: AI Personal Finance & Budgeting Platform

Complete Project Documentation, Problem Statement, System Architecture & Database Design

1. Abstract

SmartPaisa is a full-stack, AI/ML-driven personal finance management platform designed specifically for Indian college students and young professionals. With the rapid adoption of instant digital payments (UPI, QR codes, and digital wallets), young earners and students make dozens of micro-transactions daily without visibility into their cumulative monthly burn rate. SmartPaisa solves this problem by combining automated Indian merchant & UPI transaction categorization, multi-model machine learning expenditure forecasting, statistical anomaly detection, explainable financial health scoring, goal-based savings tracking, and a conversational AI Financial Advisor - all secured with Supabase PostgreSQL Row Level Security (RLS) and India DPDP Act 2023 compliance.

2. Problem Statement

2.1 Background & Core Problem

In India, over 75% of college students and early-career professionals rely on UPI (Google Pay, PhonePe, Paytm) for daily micro-transactions - ranging from INR 20 tea stalls and metro rides to food delivery (Swiggy/Zomato) and quick-commerce (Blinkit/Zepto). Because digital payments remove the psychological friction of handing over physical cash, users suffer from frictionless overspending:

* 1. Unmonitored Micro-Transactions: Small daily UPI payments accumulate into thousands of rupees by month-end without the user realizing where their allowance or salary went.
* 2. Lack of Indian Context in Global Finance Apps: Existing international budgeting tools fail to recognize Indian UPI descriptions, local merchants (IRCTC, DMRC, Jio, Apollo, Zepto), or irregular student allowance cycles.
* 3. Reactive Instead of Predictive Tracking: Traditional expense trackers only show past spending after a budget is already blown, offering no forward-looking prediction of end-of-month expenditure or daily safe-spending limits.
* 4. Financial Illiteracy & Privacy Risks: Many commercial finance apps monetize user data through predatory loan ads and lack transparent, explainable financial guidance tailored to students.

2.2 Limitations of Existing Systems vs. Proposed System (SmartPaisa)

* Transaction Categorization: Existing trackers require manual input and fail on UPI strings -> SmartPaisa provides a 3-Tier AI Categorizer recognizing 100+ Indian merchants + CSV/PDF/Receipt OCR import.
* Budget Monitoring: Existing apps use static monthly caps -> SmartPaisa computes a live Dynamic Safe Daily Allowance and proactive AI category recommendations.
* Future Forecasting: Existing apps only plot historical spend -> SmartPaisa runs 4-Model ML Time-Series Forecasting (Moving Avg, OLS, Random Forest, XGBoost) with 95% Confidence Intervals.
* Anomaly Detection: Existing apps lack statistical outlier checks -> SmartPaisa combines Isolation Forest, Local Outlier Factor (LOF), Rolling Z-Score, and 48h Duplicate Detection.
* Financial Health Scoring: Existing apps rely on credit history -> SmartPaisa computes an Explainable 0-100 Financial Health Score based on relative savings and budget adherence.
* Data Privacy & Security: Commercial apps share data with loan marketers -> SmartPaisa enforces PostgreSQL Row Level Security (RLS) and DPDP Act 2023 export/deletion controls.

3. Project Objectives

* Objective 1: Automate Indian Expense Categorization across 12 student-centric categories using a hybrid Rule + ML + Active Learning pipeline.
* Objective 2: Predict Future Expenditure over 7-day, End-of-Month, and 30-day horizons using ensemble regression models with live MAE, RMSE, and MAPE metrics.
* Objective 3: Detect Spending Anomalies Early including category spikes and accidental duplicate UPI charges within 48 hours.
* Objective 4: Provide Explainable Financial Guidance via a conversational AI Financial Assistant and a 10-module Indian Financial Literacy Centre.
* Objective 5: Enforce Strict Data Isolation guaranteeing zero cross-account data leakage via Supabase Row Level Security (auth.uid() = user_id).

4. System Architecture & Technology Stack

* Frontend Layer: React 18.3, TypeScript 5.6, Vite 5.4, React Router v6, Tailwind CSS 3.4, Framer Motion 14.9, Lucide React Icons, Recharts 2.13.
* Validation & State: React Hook Form 7.53, Zod 3.23 schema validation, AuthContext, FinanceContext, CookieContext.
* Backend & Database: Supabase Auth (JWT), Supabase PostgreSQL 15+ with Row Level Security (RLS), Supabase Edge Functions.
* AI & Machine Learning: Google Gemini API, Client TypeScript ML Engine (mlEngine.ts, aiKnowledgeEngine.ts), Python FastAPI Microservice (Scikit-Learn, XGBoost, Pandas, NumPy).

5. Core Functional Modules & Mathematical Formulations

Module 1: Authentication & DPDP Act 2023 Compliance (/login, /signup, /privacy-data)

Provides full email/password authentication via Supabase Auth, an 18+ Age Eligibility verification gate, unbundled optional marketing consent, automatic PostgreSQL profile creation trigger (on_auth_user_created), one-click JSON/CSV data export, and account deletion requests.

Module 2: Executive Financial Dashboard (/dashboard)

Renders 15+ live KPIs (Total Spend, Monthly Budget, Remaining Balance, Daily Velocity, Savings Rate, Budget Utilization %, End-of-Month Forecast, Health Score) and 6 interactive Recharts visualizations with WCAG tabular view toggles.

```
Safe Daily Spend = max(0, (Monthly Budget - Spent So Far) / Remaining Days in Month)
```

Module 3: 3-Tier AI Expense Categorization (/expenses)

Classifies transactions across 12 categories (Food & Dining, Groceries, Transport, Rent, Education, Shopping, Entertainment, Bills & Recharge, Subscriptions, Health, Personal Care, Other) using: (1) Active Learning user feedback (0.99 confidence), (2) Deterministic Indian Merchant & UPI Keyword Rule Engine (100+ merchants like Swiggy, Zomato, Blinkit, Zepto, Uber, DMRC, IRCTC, Jio, Airtel), and (3) TF-IDF + Logistic Regression classifier.

Module 4: Multi-Model Time-Series Expense Forecasting (/predictions)

Evaluates 4 chronological models using walk-forward validation without future-data leakage: Moving Average, Linear Regression (OLS), Random Forest Regressor, and XGBoost Gradient Boosting.

```
MAE = (1/n) Sum |y - y_hat|   |   RMSE = sqrt((1/n) Sum (y - y_hat)^2)   |   MAPE = (100%/n) Sum |(y - y_hat)/y|
```

Module 5: Explainable Financial Health Score & Anomaly Detection (/insights)

```
Health Score (0-100) = 0.30(Budget Adherence) + 0.25(Savings Consistency) +
0.20(Expense Stability) + 0.15(Emergency Fund) + 0.10(Goal Progress)
```

Anomaly detection combines Isolation Forest, Local Outlier Factor (LOF k-NN density), Rolling Category Z-Score (Z = (x - mean)/std > 2.5), and 48-hour duplicate transaction detection.

Module 6: Smart Budgets, Savings Goals & Statement Scanner (/budgets, /savings, /import)

Recommends category budget caps based on historical velocity; tracks savings goals with strict separation between Verified Actual Deposits and Planned Contributions; imports CSV/PDF statements and Receipt OCR images into a validated staging table.

Module 7: Hybrid AI Financial Assistant & Literacy Centre (/ai-assistant, /learn)

Combines deterministic ledger math with Google Gemini LLM reasoning to answer affordability checks ("Can I afford a INR 2,000 purchase this week?"), month-over-month audits, and savings plans. Includes 10 interactive INR financial literacy modules and quizzes.

6. Database Schema & Row Level Security (RLS)

* 1. public.profiles: Stores user metadata (id, full_name, email, currency="INR", age_verified, marketing_consent). RLS: SELECT/UPDATE where auth.uid() = id.
* 2. public.expenses: Stores expense ledger (id, user_id, title, amount, category, expense_date, payment_method, notes). RLS: Full CRUD where auth.uid() = user_id.
* 3. public.monthly_budgets: Stores monthly budget caps (id, user_id, month, year, amount). Unique constraint on (user_id, month, year). RLS: Full CRUD where auth.uid() = user_id.
* 4. public.ai_conversations & ai_messages: Stores AI chat history per user. RLS: SELECT/INSERT/DELETE where auth.uid() = user_id.
* 5. public.privacy_requests & consent_records: Audit trail for DPDP Act data export, account deletion, and cookie/marketing consents. RLS: SELECT/INSERT where auth.uid() = user_id.

7. Software & Hardware Requirements

* Software Requirements: Windows 10/11, macOS, or Linux; Node.js >= 18.x; npm >= 9.x; Supabase PostgreSQL 15+; Python 3.10+ (optional for FastAPI ML service); Modern Web Browser.
* Hardware Requirements: Intel Core i3 / AMD Ryzen 3 or higher; Minimum 4 GB RAM (8 GB recommended); 500 MB free disk space; Active internet connection.

8. Installation & Execution Steps

```
1. Install packages: npm install   |   2. Configure .env with VITE_SUPABASE_URL,
VITE_SUPABASE_ANON_KEY, VITE_GEMINI_API_KEY   |   3. Run SQL migration in Supabase
SQL Editor (supabase/migrations/20261102000000_init_smartpaisa.sql)   |   4. Start
dev server: npm run dev   |   5. Build & Test: npm test && npm run build
```

9. Conclusion & Future Scope

Conclusion: SmartPaisa bridges the gap between passive expense logging and proactive financial intelligence for young Indians. By combining localized Indian UPI merchant recognition, ensemble ML forecasting, anomaly detection, and bank-grade Row Level Security, it empowers students to stay within budget and build sustainable savings habits.

Future Scope: (1) Real-time bank & UPI feed synchronization via India RBI Account Aggregator (AA) framework, (2) Multilingual voice-based expense logging in Hindi, Telugu, Tamil, and English, and (3) Shared hostel/flatmate bill-splitting and settlement tracking.

```
Health Score (0-100) = 0.30(Budget Adherence) + 0.25(Savings Consistency) +
0.20(Expense Stability) + 0.15(Emergency Fund) + 0.10(Goal Progress)
```

Anomaly detection combines Isolation Forest, Local Outlier Factor (LOF k-NN density), Rolling Category Z-Score (Z = (x - mean)/std > 2.5), and 48-hour duplicate transaction detection.

Module 6: Smart Budgets, Savings Goals & Statement Scanner (/budgets, /savings, /import)

Recommends category budget caps based on historical velocity; tracks savings goals with strict separation between Verified Actual Deposits and Planned Contributions; imports CSV/PDF statements and Receipt OCR images into a validated staging table.

Module 7: Hybrid AI Financial Assistant & Literacy Centre (/ai-assistant, /learn)

Combines deterministic ledger math with Google Gemini LLM reasoning to answer affordability checks ("Can I afford a INR 2,000 purchase this week?"), month-over-month audits, and savings plans. Includes 10 interactive INR financial literacy modules and quizzes.

6. Database Schema & Row Level Security (RLS)

* 1. public.profiles: Stores user metadata (id, full_name, email, currency="INR", age_verified, marketing_consent). RLS: SELECT/UPDATE where auth.uid() = id.
* 2. public.expenses: Stores expense ledger (id, user_id, title, amount, category, expense_date, payment_method, notes). RLS: Full CRUD where auth.uid() = user_id.
* 3. public.monthly_budgets: Stores monthly budget caps (id, user_id, month, year, amount). Unique constraint on (user_id, month, year). RLS: Full CRUD where auth.uid() = user_id.
* 4. public.ai_conversations & ai_messages: Stores AI chat history per user. RLS: SELECT/INSERT/DELETE where auth.uid() = user_id.
* 5. public.privacy_requests & consent_records: Audit trail for DPDP Act data export, account deletion, and cookie/marketing consents. RLS: SELECT/INSERT where auth.uid() = user_id.

7. Software & Hardware Requirements

* Software Requirements: Windows 10/11, macOS, or Linux; Node.js >= 18.x; npm >= 9.x; Supabase PostgreSQL 15+; Python 3.10+ (optional for FastAPI ML service); Modern Web Browser.
* Hardware Requirements: Intel Core i3 / AMD Ryzen 3 or higher; Minimum 4 GB RAM (8 GB recommended); 500 MB free disk space; Active internet connection.

8. Installation & Execution Steps

```
1. Install packages: npm install   |   2. Configure .env with VITE_SUPABASE_URL,
VITE_SUPABASE_ANON_KEY, VITE_GEMINI_API_KEY   |   3. Run SQL migration in Supabase
SQL Editor (supabase/migrations/20261102000000_init_smartpaisa.sql)   |   4. Start
dev server: npm run dev   |   5. Build & Test: npm test && npm run build
```

9. Conclusion & Future Scope

Conclusion: SmartPaisa bridges the gap between passive expense logging and proactive financial intelligence for young Indians. By combining localized Indian UPI merchant recognition, ensemble ML forecasting, anomaly detection, and bank-grade Row Level Security, it empowers students to stay within budget and build sustainable savings habits.

Future Scope: (1) Real-time bank & UPI feed synchronization via India RBI Account Aggregator (AA) framework, (2) Multilingual voice-based expense logging in Hindi, Telugu, Tamil, and English, and (3) Shared hostel/flatmate bill-splitting and settlement tracking.